

第11章：无监督学习

张正



哈尔滨工业大学（深圳）

zhengzhang@hit.edu.cn

<http://faculty.hitsz.edu.cn/zhangzheng>

本章内容

一、基本概念

二、k-均值聚类

三、聚类的表示

四、层次聚类

五、距离函数

六、数据标准化

七、混合属性的处理

八、聚类算法的选择

九、聚类的评估

一、基本概念

- 监督学习与无监督学习

- **监督学习**：发现数据属性和类别属性之间的关联模式，然后构建有效的关联规则

- 利用这些模式用来预测未知数据实例的类别属性

- **无监督学习**：数据没有目标属性

- 实现和发现数据中存在的内在结构

一、基本概念

- 聚类通常被称为无监督学习，在聚类中那些表示数据类别的分类或分组信息没有事先给出
- 聚类(Clustering)是一种发现数据中的相似群(聚类簇, clusters)的技术
 - 处于相同聚类簇的数据实例彼此相似，处于不同聚类簇中的实例则彼此不同
- 由于历史的原因，聚类和无监督学习的关系更加紧密，甚至被认为是同义词
 - 事实上，关联规则挖掘也是无监督学习
- 本章主要介绍聚类算法

一、基本概念

- 聚类是无监督学习最典型的学习算法
 - 聚类解决的是未知类别的训练样本预测问题
- 所谓聚类就是将对物理或抽象对象的集合分组，依据在某些方面类似的数据对象进行划分簇的过程
- 聚类生成的组称为簇，簇是数据对象的集合
 - 簇内部的任意两个对象之间具有较高的相似度
 - 属于不同簇的两个对象之间具有较高的相异度
- 相似度和相异度可以根据描述对象的属性值来计算，对象间的距离是最常采用的相异度量指标

一、基本概念

- 聚类的目的

- 聚类既能作为一个单独过程，用于找寻数据内在的分布结构，也可作为分类等其他学习任务的前驱过程
- **实例1**：根据身材把人分组的方法通常就采用聚类。T恤分“小号”、“中号”、“大号”
 - 为每个顾客量身定做：太贵
 - 仅一种型号的T恤：大多数人不合身
- **实例2**：在营销学中，对客户类型进行划分，为每组客户指定一个套营销策略，也是采用聚类来完成

一、基本概念

- **实例3**：商业应用对新用户的类型定义可能不太容易，此时往往可先对用户数据进行聚类，然后再根据聚类结果将每个簇定义为一个类，最后再基于这些类训练分类模型，用于判别新用户的类型
- **实例4**：对给定文本，需要根据它们内容的相似性来进行组织
 - ▣ 建立一个主题层次
- **事实上，聚类是数据挖掘技术中应用最广泛的技术之一**
 - ▣ 发展历史长，应用领域广泛。比如：医学类、心理学、植物学、社会学、生物学、营销学、保险、图书馆等
 - ▣ 近年来，在线文档的快速发展，文本聚类研究成为关注的重点

一、基本概念

- 聚类的概述

- 基于不同的学习策略，人们设计出多种不同类型的聚类算法，很难对这些聚类算法提出一个简洁的分类。大体上，主要的聚类算法可以分为如下五类：

- 基于划分的方法
 - 基于层次的方法
 - 基于密度的方法
 - 基于网格的方法
 - 基于模型的方法

一、基本概念

- 聚类的概述

- 距离函数（相似性或相异性）：度量两个数据点（对象）的相似程度
- 聚类评价
 - 类内差异（聚类内部距离）：最小化
 - 类间差异（聚类外部距离）：最大化
 - 指标：ACC, NMI, Purity, F1-Score, ARI, Recall, etc.
- 聚类结果的质量：与表示、算法、距离函数和应用领域有很大关系

二、 k -均值聚类

- 基于划分的方法就是采用目标函数最小化的策略，通过迭代把数据对象划分成 k 个组，每个组为一个簇
- 基于划分的方法需要满足如下两个条件：
 - 每个分组至少包含一个对象
 - 每个对象属于且仅属于某一个分组
- k -均值算法是基于划分的聚类算法。 k -均值算法根据某个距离函数反复地把数据分入 k 个聚类簇中
 - 设数据点(或实例)的集合 D 为 $\{x_1, x_2, \dots, x_n\}$ ，其中， $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$ 是实空间 $X \subseteq R^d$ 中的向量， d 表示数据的属性数目(数据空间维数)

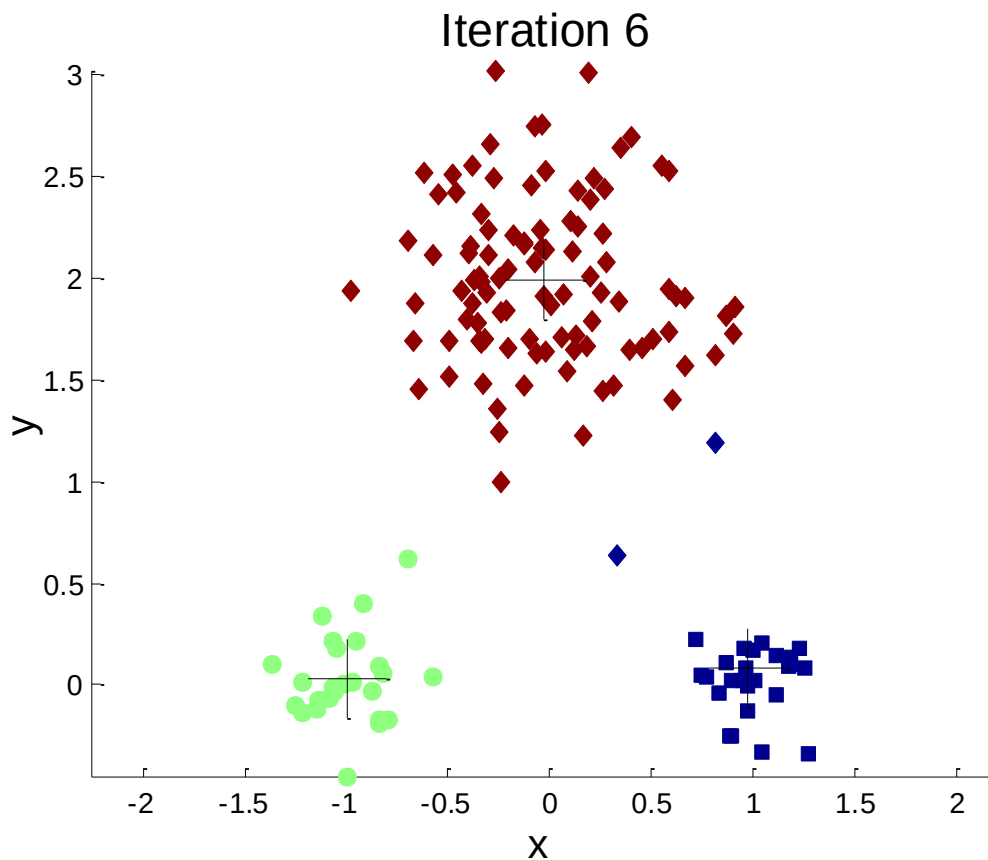
二、 k -均值聚类

- k -均值算法把给定的数据划分为 k 个聚类簇
 - 每个聚类簇中有一个聚类中心，它用来表示某个聚类，这个中心是聚类中所有数据点的**均值**
 - k 是由用户指定的
- k -均值算法
 - 给定 k ， k -均值算法执行步骤：
 - 随机选取 k 个数据点作为**初始聚类中心**
 - 计算每个数据点与各个中心之间的**距离**，把每个数据点**分配**给距离它最近的聚类中心
 - 数据点分配以后，每个聚类的**聚类中心**会根据聚类现有的数据点**重新计算**
 - 这个过程将**不断重复**直到满足某个终止条件为止

二、 k -均值聚类

- k -均值算法

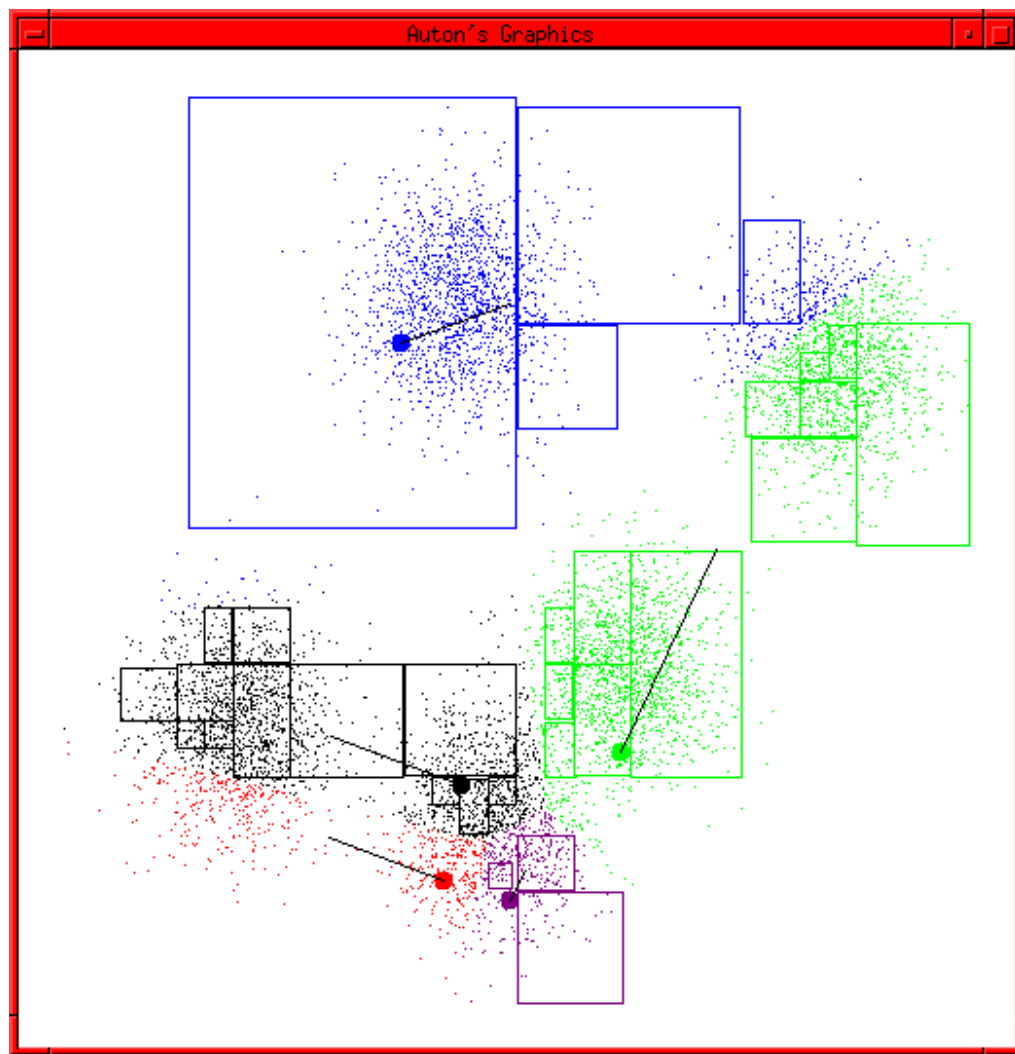
演示



二、 k -均值聚类

● k -Means举例

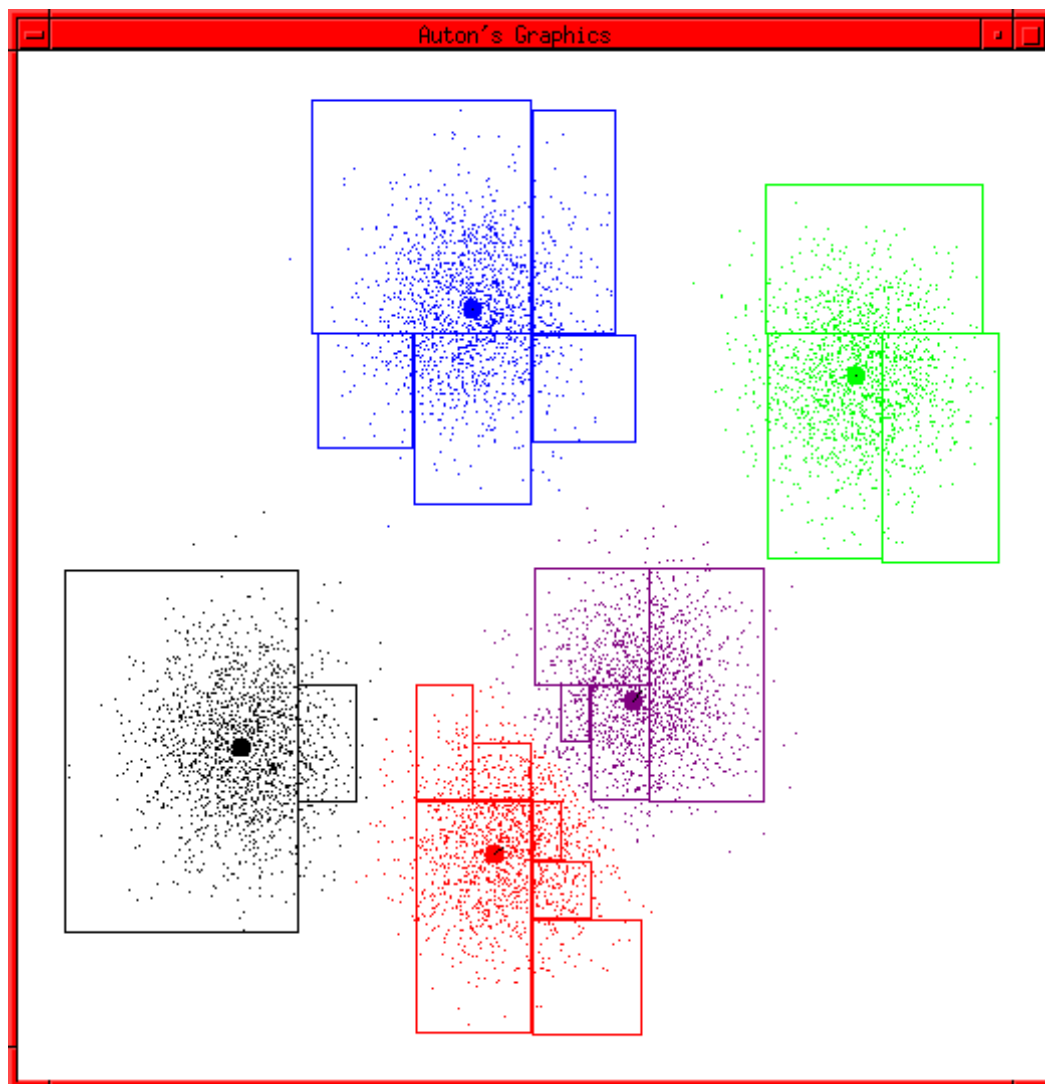
1. 设置聚类个数 k (e.g. $k=5$)
2. 随机猜测 k 个簇的类中心位置
3. 对每一个样本点寻找其最近的类中心
4. 对于每一个簇中心寻找所在簇样本点的中心
5. 重复直到收敛



二、 k -均值聚类

● k -Means举例

1. 设置聚类个数 k (e.g. $k=5$)
2. 随机猜测 k 个簇的类中心位置
3. 对每一个样本点寻找其最近的类中心
4. 对于每一个簇中心寻找所在簇样本点的中心
5. 重复直到收敛



二、 k -均值聚类

- 算法内容

k -means (k, D)

选择 k 个数据点作为初始聚类中心;

Repeat

For 对于每一个数据点 $x \in D$:

 计算 x 到任意一个聚类中心的距离;

 将 x 划分到离它最近的聚类中心所属的类;

Endfor

利用当前的划分结果重新计算聚类中心;

Until 满足终止条件时停止

k 值怎么选?

如何初始化?

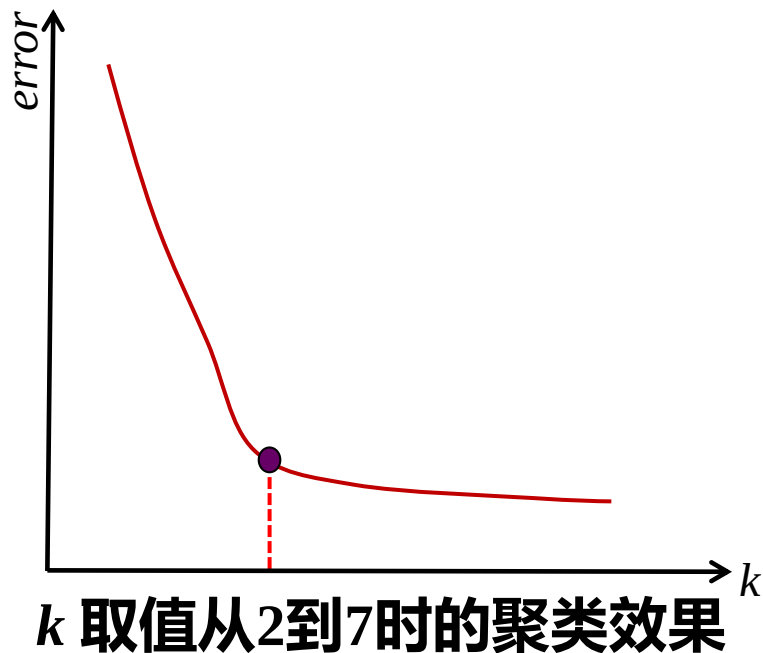
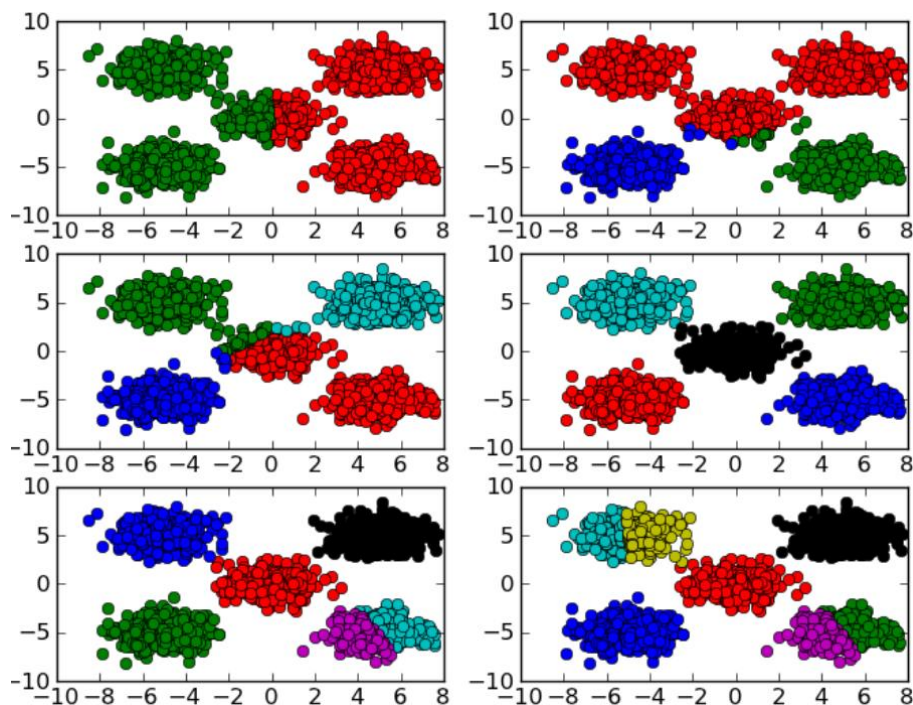
距离怎么计算?

什么时候终止?

二、 k -均值聚类

● 关于聚类的 k 值选择

- 给定一个合适的类簇指标，比如平均半径或直径，只要我们假设的类簇的数目等于或者高于真实的类簇的数目时，该指标上升会很缓慢，而一旦试图得到少于真实数目的类簇时，该指标会急剧上升



二、 k -均值聚类

● 关于聚类的初始划分

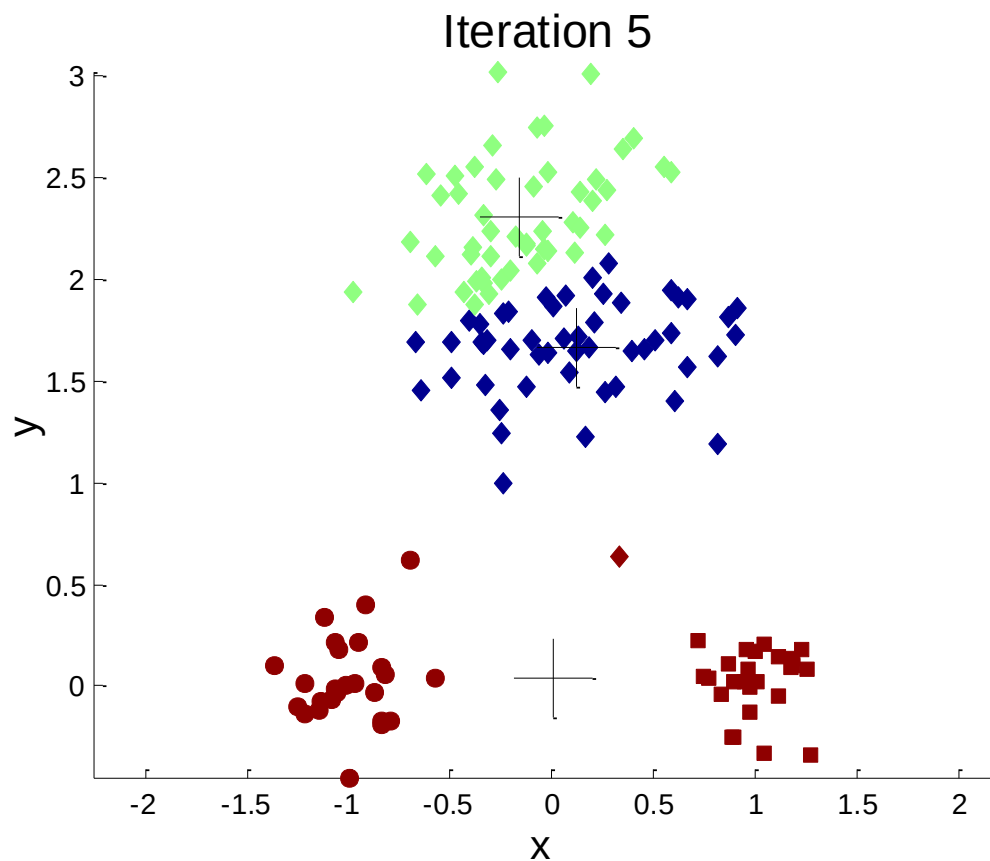
初始划分：一般选择有代表性的点，再进行初始聚类。

代表性选择方法：

- 经验选择：问题的性质、数据分布，直观上合理
- 随机分成 k 个簇，选各个簇的重心作为代表点
- “密度”法：计算每个样本的一定球形邻域内的样本数作为“密度”，选“密度”最大的样本点作为第一个代表点，在离它一定距离之外选最大“密度”点作为第二个代表点，...，依次类推
- 用前 k 个样本点作为代表点
- 用 $k-1$ 聚类求 k 个代表点：各类中心外加离它们最远的样本点，从1类开始

二、 k -均值聚类

- 关于聚类的初始划分



二、k-均值聚类

● 关于聚类的距离计算

- 在那些均值能被定义和计算的数据集上，均能使用k-均值算法
- 在欧式空间，聚类均值可以使用如下公式经验选择：问题的性质、数据分布，直观上合理

$$m_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

- 数据点与聚类中心的距离使用如下公式：

$$\begin{aligned} \text{dist}(x_i, m_j) &= \|x_i - m_j\| \\ &= \sqrt{(x_{i1} - m_{j1})^2 + (x_{i2} - m_{j2})^2 + \cdots + (x_{ir} - m_{jr})^2} \end{aligned}$$

二、 k -均值聚类

● 关于聚类的终止条件

- 没有（或最小数目）数据点被重新分配给不同的聚类
- 没有（或最小数目）聚类中心再发生变化
- 误差平方和（Sum of Squared Error , SSE）局部最小

$$SSE = \sum_{j=1}^k \sum_{x \in C_j} dist(x, m_j)^2$$

其中， C_j 表示第 j 个聚类簇， m_j 是聚类 C_j 的聚类中心，一般定义在第 C_j 簇中所有数据点的均值向量， $dist(x, m_j)$ 表示数据点 x 和聚类中心 m_j 之间的距离。

二、 k -均值聚类

- k -均值算法应用

K=2



K=3



K=10



Original



4%



8%



17%



二、 k -均值聚类

● 算法举例：

下面给出一个样本事务数据库，并对它实施 k -均值算法。

序号	属性1	属性2
1	1	1
2	2	1
3	1	2
4	2	2
5	4	3
6	5	3
7	4	4
8	5	4

设 $n=8$, $k=2$, 执行下面的步骤：

➤ 第一次迭代

- 假定随机选择的两个对象，如序号1和序号3当作初始点，分别找到离两点最近的对象，并产生两个簇{1, 2}和{3, 4, 5, 6, 7, 8}
- 对于产生的簇分别计算平均值，得到平均值点：对于{1, 2}，平均值点(1.5,1)；对于{3, 4, 5, 6, 7, 8}，平均值点(3.5,3)

二、 k -均值聚类

➤ 第二次迭代：

- 通过平均值调整对象所在的簇，重新聚类，即将所有点按离平均值点 $(1.5,1)$ 和 $(3.5,3)$ 最近的原则重新分配。得到两个簇： $\{1,2,3,4\}$ 和 $\{5,6,7,8\}$
- 重新计算簇平均值点，得到新的平均值点为： $(1.5,1.5)$ 和 $(4.5,3.5)$

➤ 第三次迭代：

- 通过平均值调整对象所在的簇，重新聚类，即将所有点按离平均值点 $(1.5,1.5)$ 和 $(4.5,3.5)$ 最近的原则重新分配。得到两个簇： $\{1,2,3,4\}$ 和 $\{5,6,7,8\}$
- 发现没有出现重新分配，准则函数收敛，程序结束

二、 k -均值聚类

下表给出了该例子整个过程中平均值计算和簇生成的过程和结果：

迭代次数	平均值簇1	平均值簇2	产生的新簇	新平均值簇1	新平均值簇2
1	(1, 1)	(1, 2)	{1, 2}, {3, 4, 5, 6, 7, 8}	(1.5, 1)	(3.5, 3)
2	(1.5, 1)	(3.5, 3)	{1, 2, 3, 4}, {5, 6, 7, 8}	(1.5, 1.5)	(4.5, 3.5)
3	(1.5, 1.5)	(4.5, 3.5)	{1, 2, 3, 4}, {5, 6, 7, 8}	(1.5, 1.5)	(4.5, 3.5)

二、 k -均值聚类

- k -均值算法的硬盘版本

- k -均值可以执行硬盘上的数据

- 每一次循环，算法扫描数据一次

- 聚类中心可以在每次循环中增量计算

- k -均值在处理大规模数据时（内存不足以容纳这些数据）十分有用

- 需要控制循环次数

- 实际应用中，需要设置一个限制次数(<50)

二、 k -均值聚类

● k -均值硬盘版算法

Disk- k -means(k, D)

选择 k 个数据点 $m_j (j = 1, \dots, k)$ 作为初始聚类中心

Repeat

$s_j = \mathbf{0}, j = 1, \dots, k$ % $\mathbf{0}$ 是一个全0的向量

$n_j = 0, j = 1, \dots, k$ % n_j 是第 j 个簇中样本点的数量

For 对于每一个数据点 $x \in D$:

$j = \arg \min_j \text{dist}(x, m_j)$

将 x 分到第 j 个聚类簇

$s_j = s_j + x; n_j = n_j + 1$

Endfor

计算簇中心 $m_j = \frac{s_j}{n_j}, i = 1, \dots, k$

Until 满足终止条件时停止

二、 k -均值聚类

- k -均值算法的优势与劣势

- 优势

- 简单：容易被理解，同时也容易被实现。
- 效率：时间复杂度为 $O(tkn)$

其中， n 是数据点个数； k 是聚类的个数； t 是循环次数

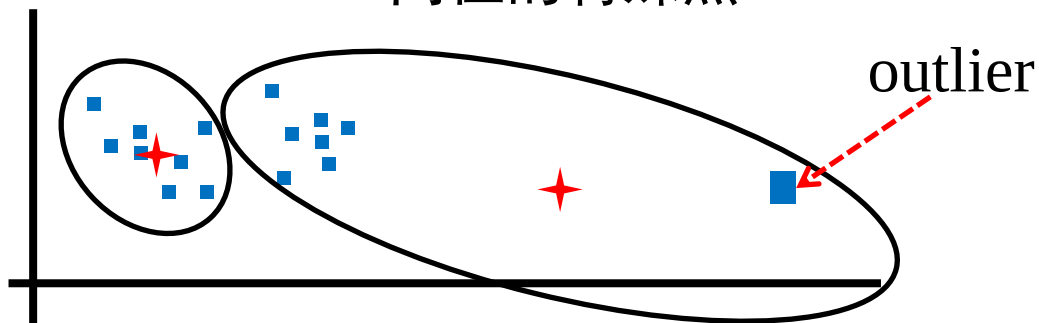
- 由于 k 和 t 通常都远远小于 n ， k -均值算法被认为相对于数据点的数目来说是线性的

- k -均值算法是聚类算法中最流行的一种算法

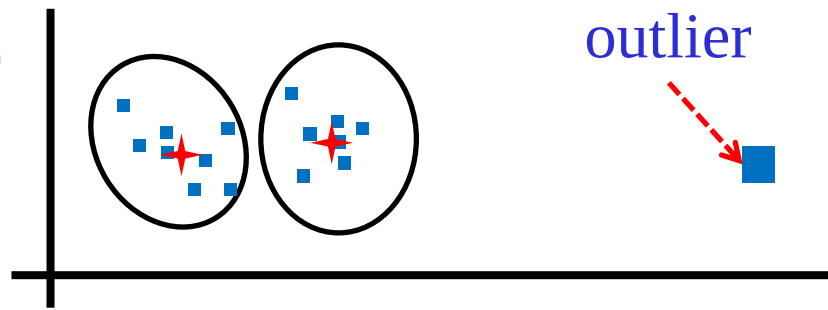
二、 k -均值聚类

➤ 劣势

- 算法只能用于那些均值能够被定义的数据集上
 - 对于范畴数据，有一种 k -均值算法的变体—— k -模算法，用于聚类范畴数据；再如名词性属性数据
- 用户需要事先指定聚类数目 k ，难以估计
- 算法对于噪声和异常值(孤立点)十分敏感
 - 异常值是指数据中那些与其他数据点相隔很远的数据点
 - 异常值可能是数据采集时产生的错误或者一些具有不同值的特殊点



(A) Undesirable clusters



(B) Ideal clusters

二、 k -均值聚类

➤ 解决异常值问题

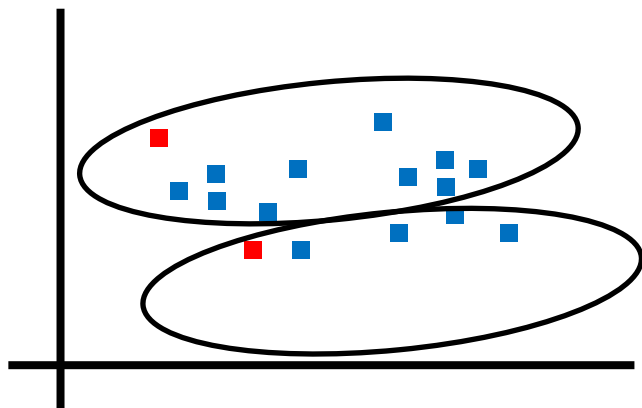
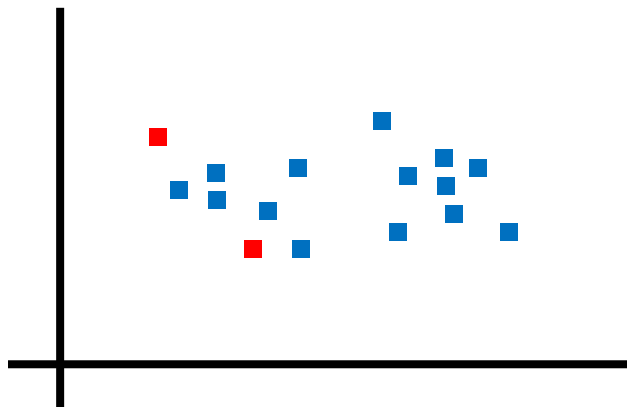
- 一种方法：在聚类过程中**去除**那些比其他任何数据点离聚类中心都要远的数据点
 - ✓ 安全起见，在**多次循环中监控**这些潜在的异常值，然后再决定是否删除它们
- 另一个方法：**随机采样**。因为在采样过程中，我们仅只选择很少一部分的数据点，因此选中异常值的概率将会小很多
 - ✓ 先用采样点进行**预先聚类**，然后把其他数据点分配给这些聚类（剩下的数据点分配给那些距离它最近的聚类中心/分类）

二、 k -均值聚类

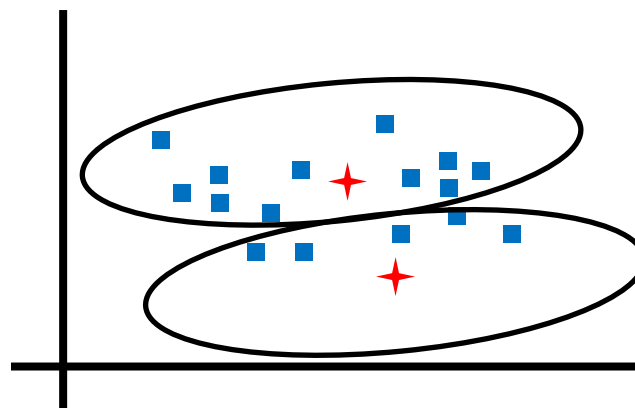
➤ 劣势（续）

- 算法对于初始种子十分敏感

(a) 随机选择种子点(即 centers)



(b) Iteration 1

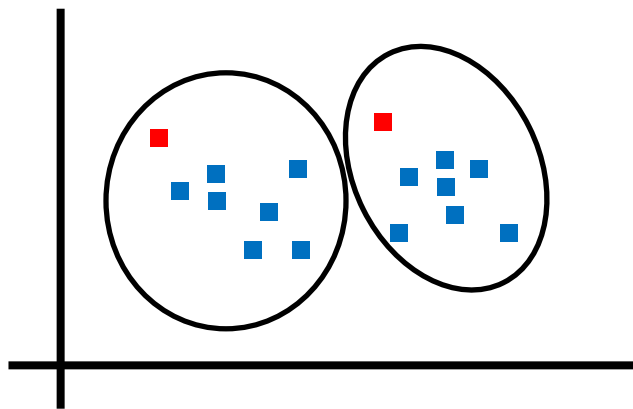
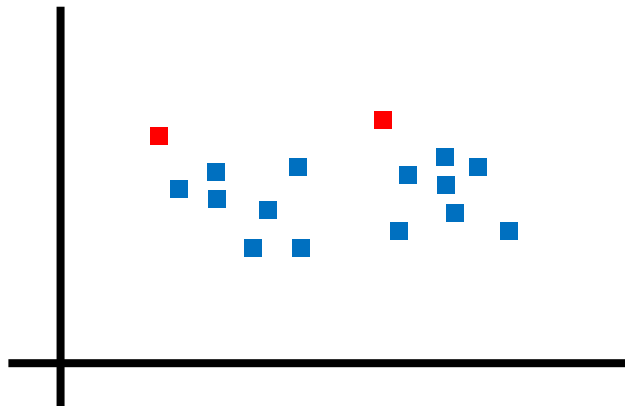


(c) Iteration 2

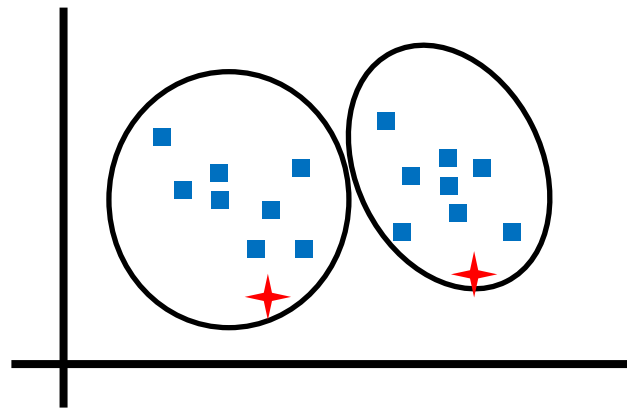
二、 k -均值聚类

- 如果选择不同的种子：比较好的结果

(a) 随机选择种子点 (即centers)



(b) Iteration 1

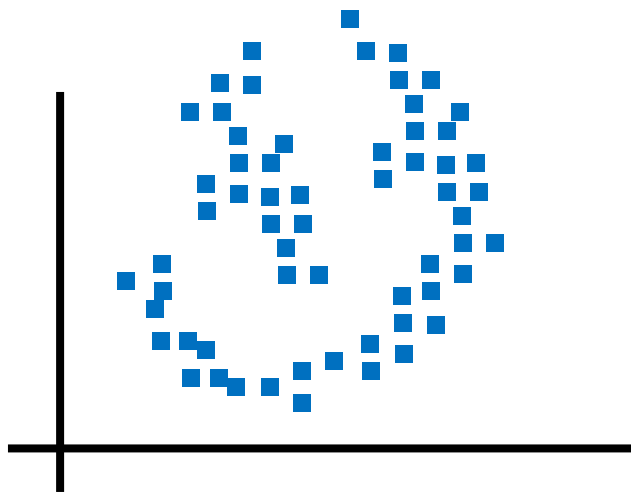


(c) Iteration 2

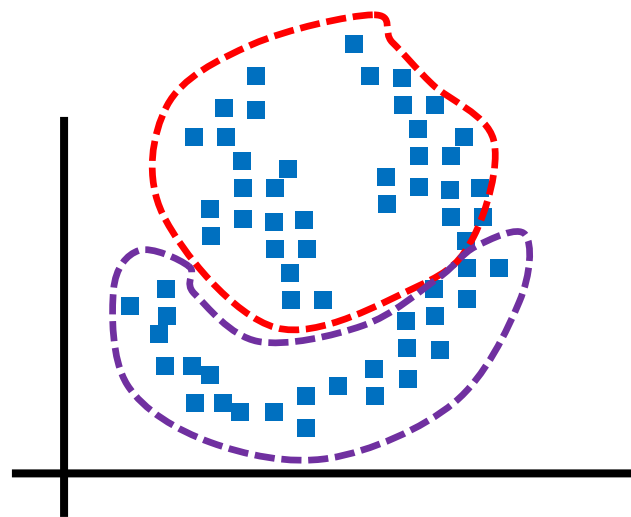
二、 k -均值聚类

➤ 劣势（续）

- ❑ 算法不适合发现那些形状不是超维椭圆体（或超维球体）的聚类



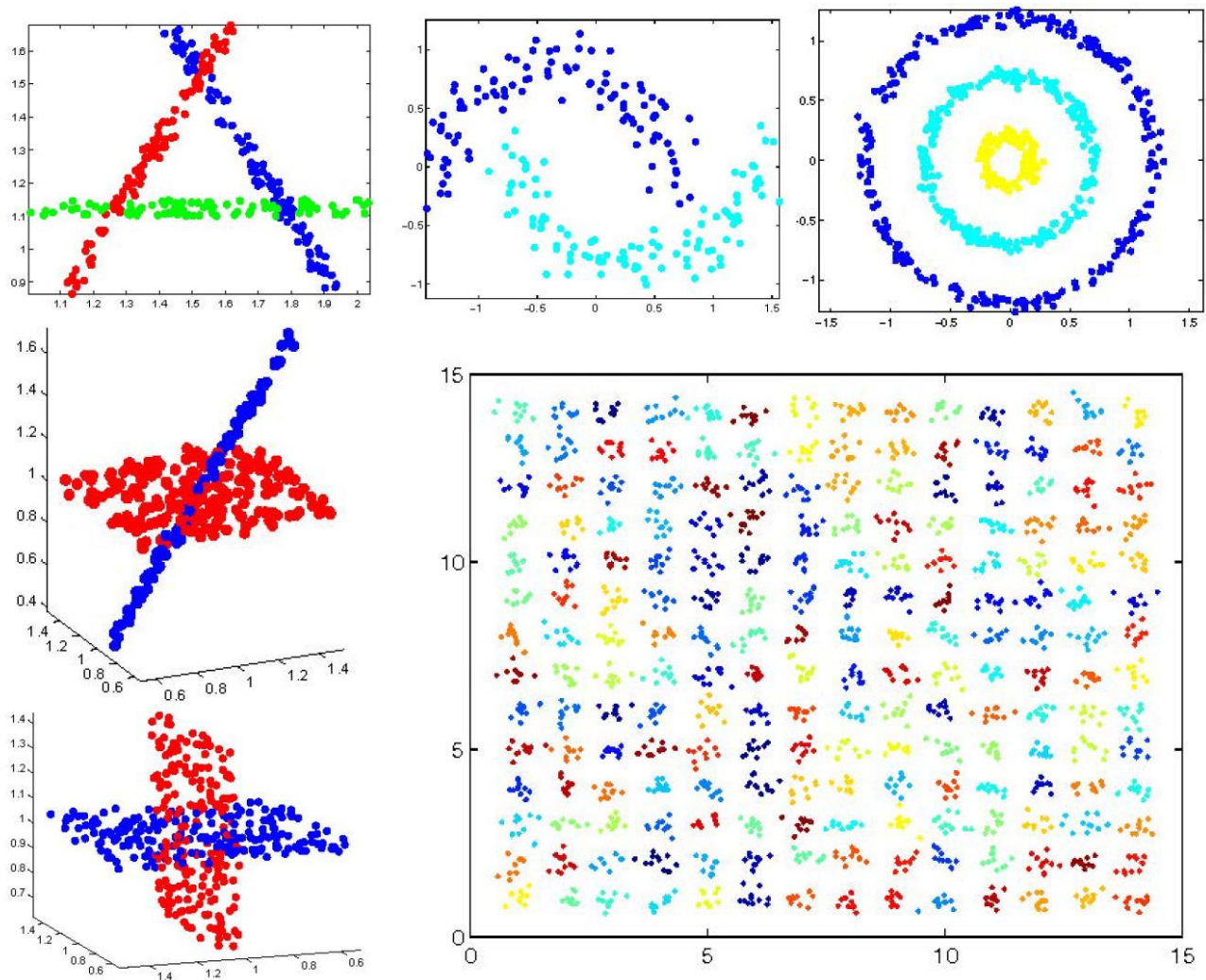
(a) Two natural clusters



(b) k -means clusters

二、 k -均值聚类

- 算法不适合发现那些形状不是超维椭圆体（或超维球体）的聚类



二、 k -均值聚类

- k -均值算法总结

- 虽然 k -均值算法有不足，但它仍是在实践中采用最为广泛的算法
 - 其它聚类算法也有它们的一系列不足之处
- 没有直接的证据证明哪一种算法在整体表现上优于 k -均值算法
 - 虽然其它聚类算法在某些特殊数据下的表现优于 k -均值算法
- 比较不同聚类算法的优劣是一个很难的任务，没有人知道正确的聚类结果是什么

三、聚类的表示

- 对于数据中的聚类需要寻找一种方法来表示这些聚类簇
- 某些应用中，只需直接输出聚类簇中的数据点即可
- 而有的应用，特别是决策相关应用中，聚类结果需要用一种压缩和可理解的方式表示
- 有利于对聚类结果的评价

三、聚类的表示

● 聚类的一般表示方法

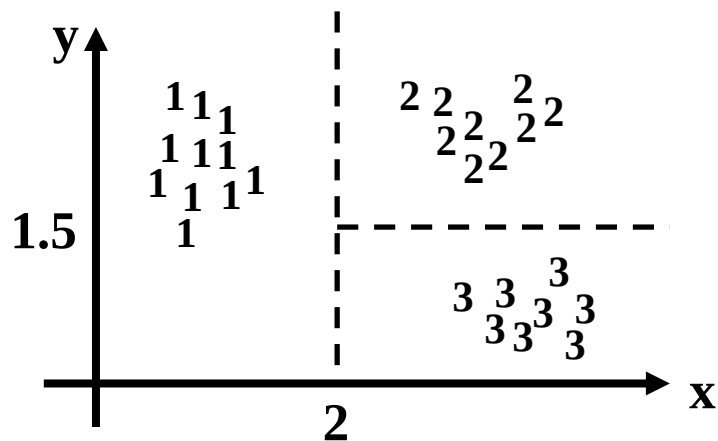
- 用**聚类中心**来表示每个聚类簇是使用最广泛的聚类表示方法
 - 计算**聚类簇的半径**和**标准差**来确定聚类在各个维度上的伸展度
 - **聚类中心表示法**对于那些高维球体形状的聚类来说已经足够
 - 如果聚类被拉长了或者是其他形状的话，聚类中心表示就可能不太适合

三、聚类的表示

➤ 利用**分类模型**来表示聚类

□ 同一聚类中的所有数据点被看作是具有**相同类别标识**，如聚类标识

✓ 通过在数据上执行某个监督学习算法来发现分类模型



$x \leq 2$: cluster 1

$x > 2, y > 1.5$: cluster 2

$x > 2, y < 1.5$: cluster 3

第11章：无监督学习

张正



哈尔滨工业大学（深圳）

zhengzhang@hit.edu.cn

<http://faculty.hitsz.edu.cn/zhangzheng>

复习

- 基本概念

- 监督学习和无监督学习(聚类)

- k -均值聚类算法

- 算法流程
- k 值选择、初始划分、距离计算、终止条件
- 例子
- k -均值聚类的硬盘版本
- k -均值聚类算法的优缺点

- 聚类的表示

- 一般表示方法：聚类中心、分类模型

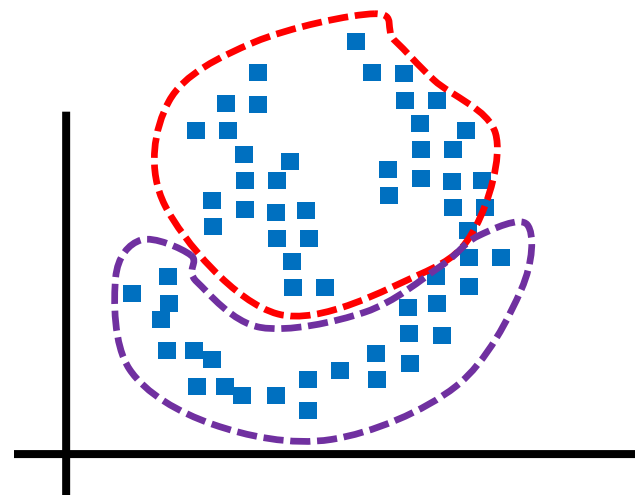
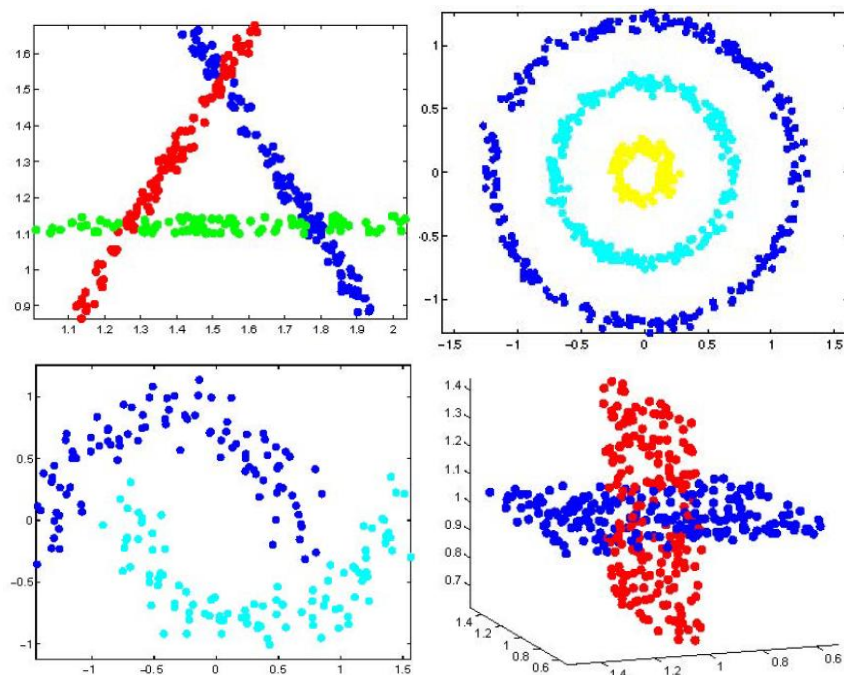
三、聚类的表示

- 利用聚类中**最为常见的值**来表示聚类
 - 这种方法通常在对**范畴属性**进行聚类时采用（k-模聚类）
 - 它同样是文本聚类中的重要方法，比如：使用一个较小的集合：每个类中高频词来表示这个类

三、聚类的表示

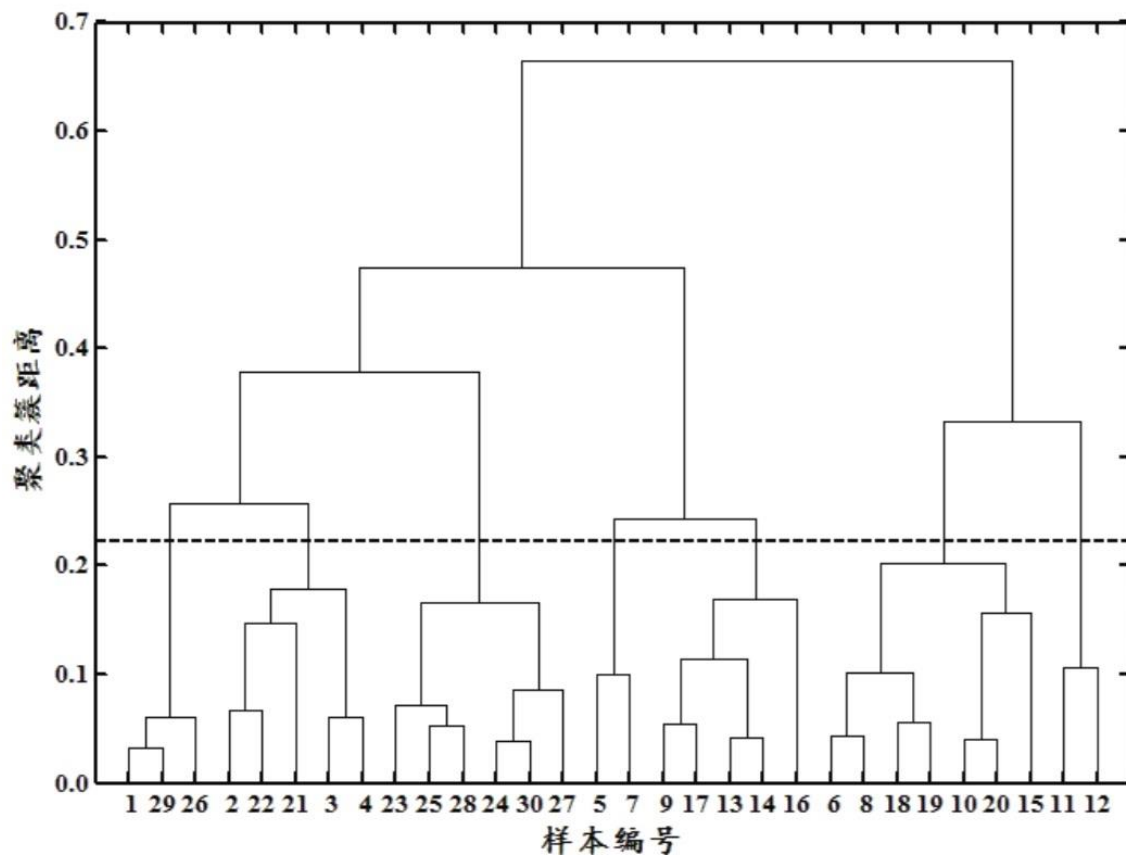
● 任意形状的聚类

- 超维椭圆体或超维球体形状的聚类容易使用聚类中心以及聚类的伸展度（标准差）、规则或者它们的组合来表示
- 任意形状的聚类是很难用它们来表示的。（一般分别输出每个聚类中的数据点）



四、层次聚类

- 层次聚类试图在不同层次对数据集进行划分，从而形成树形的聚类结构
- 生成一系列嵌套的**聚类树**（也叫**树状图**）



四、层次聚类

- 层次聚类的两种方法

- **合并（自下而上）聚类**：从树状图的最底层开始构建聚类树，将每个样本看作一个初始聚类簇
 - 合并最相似（距离最近）的两个聚类簇，形成上一层中的聚类，该过程不断重复
 - 整个过程当全部数据点都合并到一个聚类簇中时停止
- **分裂（自上而下）聚类**：从一个包含全部数据点的聚类开始
 - 根结点聚类分裂成一些子聚类簇，每个子聚类簇再递归地继续向下分裂
 - 直到出现只包含一个数据点的单节点聚类出现时停止

四、层次聚类

- 合并聚类

- 它比分裂聚类算法应用得更广泛
 - 起初，每一个数据点形成一个初始聚类（或叫结点）
 - 合并具有最短距离的两个聚类/结点
 - 继续合并
 - 直到所有结点都在一个聚类簇中为止

四、层次聚类

- 合并聚类算法

Agglomerative(D):

将每个D中的每一个数据点视作一个簇；

计算 $x_1, x_2, \dots, x_n \in D$ 中任意一对样本点之间的距离；

重复：

{

找到两个距离最近的簇；

将这两个簇合并成一个新的簇 c；

计算聚类簇c 与其它所有簇的距离；

}

直到只剩下一个簇

四、层次聚类

● 合并聚类算法

初始所有数据
点作为候选簇

计算簇与簇之
间的距离

合并簇，并对距离
矩阵更新

计算聚类簇 j 与其
它所有簇的距离

输入: 样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;
聚类簇距离度量函数 $d \in \{d_{\min}, d_{\max}, d_{\text{avg}}\}$;
聚类簇数 k .

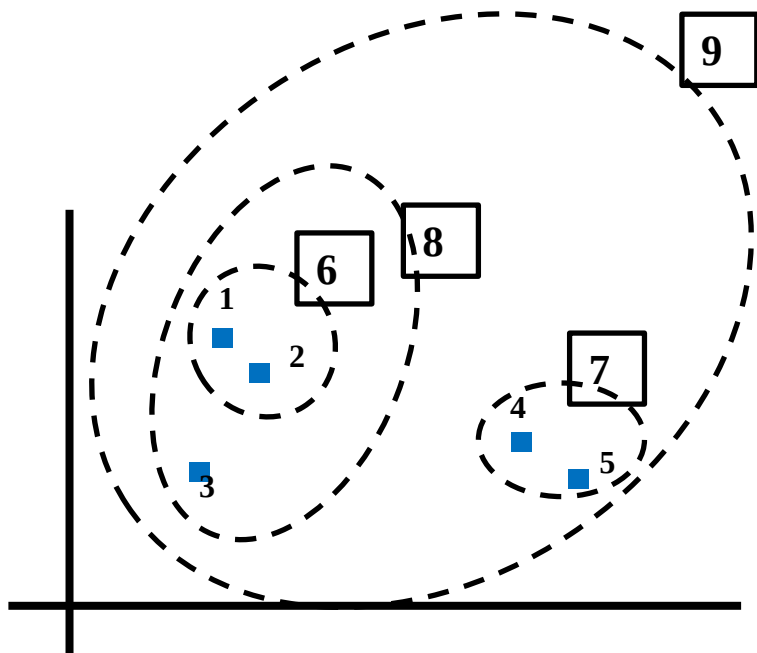
过程:

```
1: for  $j = 1, \dots, m$  do
2:    $C_j = \{\mathbf{x}_j\}$ 
3: end for
4: for  $i = 1, \dots, m$  do
5:   for  $j = i, \dots, m$  do
6:      $M(i, j) = d(C_i, C_j)$ ;
7:      $M(j, i) = M(i, j)$ 
8:   end for
9: end for
10: 设置当前聚类簇个数:  $q = m$ 
11: while  $q > k$  do
12:   找出距离最近的两个聚类簇( $C_{i^*}, C_{j^*}$ );
13:   合并( $C_{i^*}, C_{j^*}$ ):  $C_{i^*} = C_{i^*} \cup C_{j^*}$ ;
14:   for  $j = j^* + 1, \dots, q$  do
15:     将聚类簇  $C_j$  重编号为  $C_{j-1}$ 
16:   end for
17:   删除距离矩阵  $M$  的第  $j^*$  行与第  $j^*$  列;
18:   for  $j = 1, \dots, q - 1$  do
19:      $M(i^*, j) = d(C_{i^*}, C_j)$ ;
20:      $M(j, i^*) = M(i^*, j)$ 
21:   end for
22:    $q = q - 1$ 
23: end while
24: return 簇划分结果
```

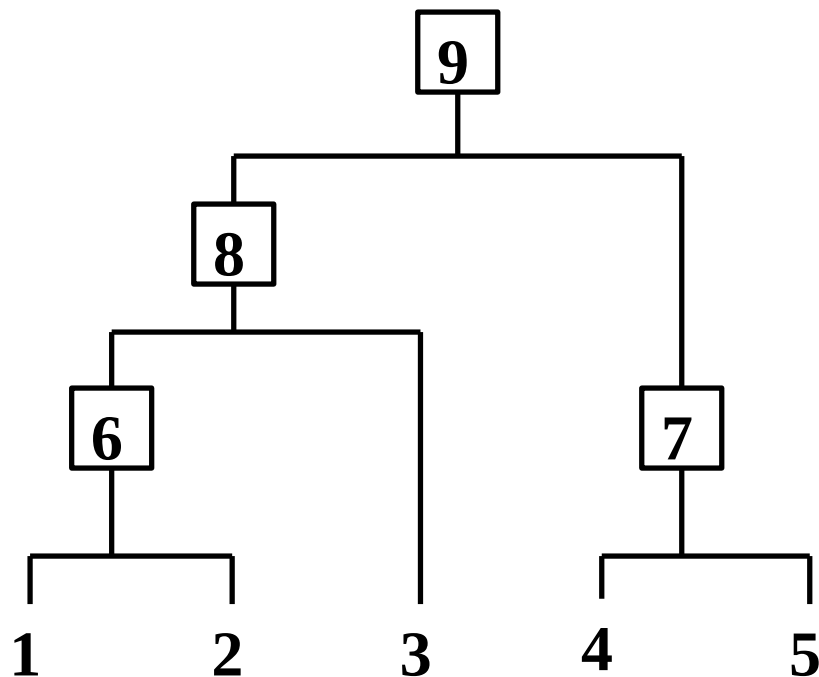
输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

四、层次聚类

● 算法举例



(A) 嵌套的聚类簇



(B) 树状图显示

四、层次聚类

下面给出一个样本事务数据库，并对它实施合并聚类算法。

序号	属性1	属性2
1	1	1
2	2	1
3	1	2
4	2	2
5	4	3
6	5	3
7	4	4
8	5	4

设 $n=8$ ，用户输入的终止条件为两个簇，执行下面的步骤：

- 初始簇：
 $\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}$
- 第一步：根据初始簇计算每个簇之间的距离，随机找出距离最小的两个簇，进行合并，最小距离为1，合并后，1，2点合并为一个簇。

四、层次聚类

- 第二步：对第一步合并后的簇计算簇间距离，找出距离最近的两个簇进行合并（合并聚类算法中计算两个簇间的相似度可由这个两不同簇距离最近的数据点对的相似度来确定）。经计算，选择最近的簇距离1，确定{3}{4}合并为一簇。
- 第三步：按第二步的操作方式，对{1,2}{3,4}{5}{6}{7}{8}计算6个簇间距离，找出距离最近的两个簇进行合并。经计算，选择最近的簇距离1，确定{5}{6}合并为一簇。

四、层次聚类

- 第四步：同样按第二步的操作， $\{7\}\{8\}$ 合并为一簇，得到 $\{1,2\}\{3,4\}\{5,6\}\{7,8\}$
- 第五步：分别计算簇之间的距离，合并 $\{1,2\}\{3,4\}$ 为一簇（两簇中， $\{1\}\{3\}$ 的距离为1，即可合并）
- 第六步：分别计算簇 $\{1,2,3,4\}\{5,6\}\{7,8\}$ 的距离，合并 $\{5,6\}\{7,8\}$ 为一簇，得到 $\{1,2,3,4\}\{5,6,7,8\}$ 两个簇，由于数目已经达到了用户输入的条件，程序结束。

四、层次聚类

下面表格给出了该例子整个过程中簇间距离计算和簇合并的过程和结果：

步骤	最近的簇距离	最近的两个簇	合并后的新簇
1	1	{1} {2}	{1,2} {3} {4} {5} {6} {7} {8}
2	1	{3} {4}	{1,2} {3,4} {5} {6} {7} {8}
3	1	{5} {6}	{1,2} {3,4} {5,6} {7} {8}
4	1	{7} {8}	{1,2} {3,4} {5,6} {7,8}
5	1	{1,2} {3,4}	{1,2,3,4} {5,6} {7,8}
6	1	{5,6} {7,8}	{1,2,3,4} {5,6,7,8}

四、层次聚类

- 两个聚类之间的距离计算

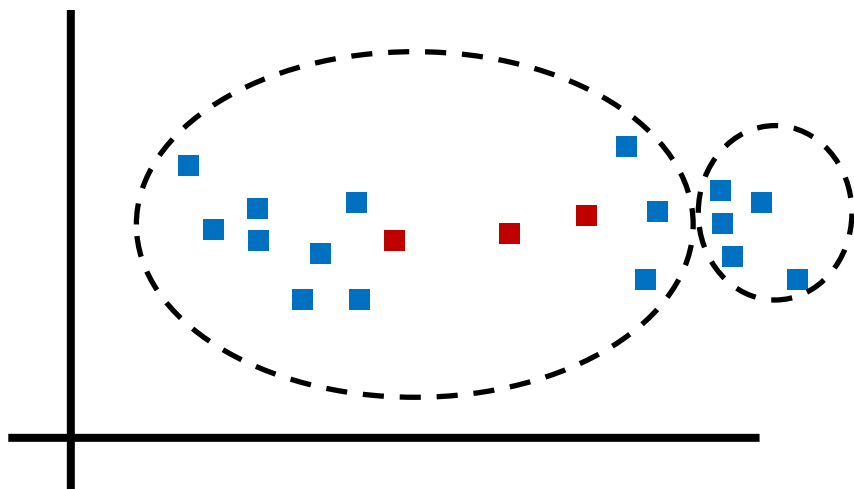
- 层次聚类中计算两个聚类簇之间的距离方法有很多
- 在不同的算法中可以用不同方法去确定两个聚类之间的距离

- 单链接方法 $d_{min}(C_i, C_j) = \min_{x \in C_i, z \in C_j} dist(x, z)$
- 全链接方法 $d_{max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} dist(x, z)$
- 平均链接方法 $d_{avg}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{z \in C_j} dist(x, z)$
- 聚类中心方法
- ...

四、层次聚类

➤ 单链接方法

- 两个聚类簇之间的距离是**两个聚类簇中距离最近的两个数据点**（来自不同聚类簇的数据点）之间的距离
- 单链接方法**适合于寻找那些形状怪异的聚类**，但是它对数据中的噪声非常敏感

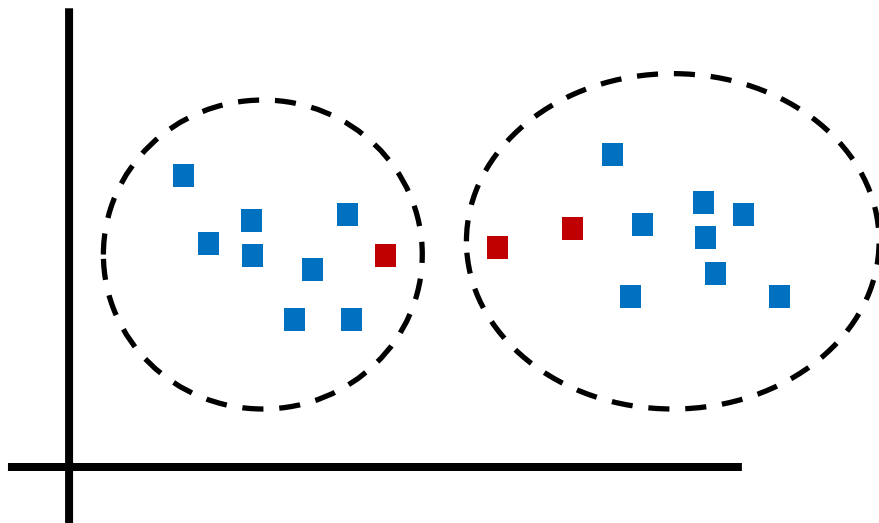


连接两个自然聚类簇存在噪声数据点，使得其中一个自然聚类被分裂

四、层次聚类

➤ 全链接方法

- 两个聚类簇之间的距离是两个聚类簇中所有数据点之间距离的**最大值**
- 全链接方法对异常值十分敏感



四、层次聚类

- **平均链接方法**：一种折衷方法
 - 介于全链接方法对于异常值的敏感性和单链接方法形成长链（这种长链不符合聚类是紧密的椭圆体对象这一常识）的趋势之间的折衷方法
 - 两个聚类簇之间的距离是两个聚类簇之中多个数据点对之间距离之和的平均值
- **聚类中心方法**：两个聚类簇之间的距离是两个聚类簇中心之间的距离

四、层次聚类

● 复杂度

- 层次聚类算法中至少有 $O(n^2)$ 的计算复杂度，其中 n 为数据点的个数
- 单链接的计算复杂度为 $O(n^2)$
- 全链接和平均链接算法的复杂度达到了 $O(n^2 \log n)$
- 层次聚类算法在处理大规模数据时十分低效
 - 采样
 - 向大规模数据扩展的方法（比如：BIRCH）

五、距离函数

- 距离计算是很多学习任务的核心技术
- 距离或者相似度函数在聚类算法中扮演者十分重要的角色
- 有许多不同的距离函数用于
 - 不同数据类型
 - 数值型数据
 - 布尔型数据
 - 符号属性
 - 不同的应用领域
 - 文本文档

五、距离函数

● 数值的属性

- 最常用的距离函数是欧几里德距离(Euclidean distance)和曼哈顿距离(Manhattan distance)
- 用 $\text{dist}(x_i, x_j)$ 来表示 d 维空间中的两个数据点之间的距离
- 这两种函数是闵可夫斯基距离(Minkowski distance)的特殊情况

$$\text{dist}(x_i, x_j) = \left(\sum_{k=1}^d |x_{ik} - x_{jk}|^p \right)^{\frac{1}{p}}$$

p : 一般是整数

p=2: 欧氏距离 (Euclidean distance)

p=1: 曼哈顿距离 (Manhattan distance)

五、距离函数

➤ 欧几里德距离和曼哈顿距离

- 如果 $p=2$, 就得到了欧几里德距离(Euclidean distance)

$$\text{dist}(x_i, x_j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{id} - x_{jd})^2}$$

- 如果 $p=1$, 就得到了曼哈顿距离(Manhattan distance)

$$\text{dist}(x_i, x_j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{id} - x_{jd}|$$

- 加权欧几里德距离

$$\text{dist}(x_i, x_j) = \sqrt{w_1(x_{i1} - x_{j1})^2 + w_2(x_{i2} - x_{j2})^2 + \dots + w_d(x_{id} - x_{jd})^2}$$

五、距离函数

➤ 平方欧几里德距离和切比雪夫距离

- 平方欧几里德距离(Euclidean distance)：标准欧几里德距离平方，加大了距离较远的数据点的权重

$$dist(x_i, x_j) = (x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{id} - x_{jd})^2$$

- 切比雪夫距离(Chebychev distance)：当需要定义当两个数据点有一个属性值不同时它们就是“不同”的，就用该距离公式

$$dist(x_i, x_j) = \max(|x_{i1} - x_{j1}|, |x_{i2} - x_{j2}|, \dots, |x_{id} - x_{jd}|)$$

五、距离函数

- 布尔属性

- 布尔属性：具有两个状态或值，比如：性别中只有男和女
- 使用混淆矩阵来介绍这种距离度量方法
- 给定第 i 个和第 j 个数据点，即 x_i 和 x_j

五、距离函数

➤ 混淆矩阵

二元属性的列联表				
对象 i	对象 j			
		1	0	sum
	1	a	b	a+b
	0	c	d	c+d
	sum	a+c	b+d	a+b+c+d

- **a:** x_i, x_j 两个数据点中, 具有相同属性值1的属性个数
- **b:** x_i, x_j 两个数据点中满足 $x_{if} = 1$ 且 $x_{jf} = 0$ 条件的属性个数, 其中 x_{if} x_{jf} 分别表示 x_i, x_j 两个数据点的第 f 个属性值
- **c:** x_i, x_j 两个数据点中满足 $x_{if} = 0$ 且 $x_{jf} = 1$ 条件的属性个数, 其中 x_{if} x_{jf} 分别表示 x_i, x_j 两个数据点的第 f 个属性值
- **d:** x_i, x_j 两个数据点中具有相同属性值0的属性个数

五、距离函数

➤ 对称布尔属性

- 当布尔属性的两个状态具有相同的重要性并且具有相同的权重时，我们称这个布尔属性是**对称的**。如：性别属性，男和女
- 对称属性中使用普遍的距离函数**简单匹配距离**(Simple Matching Coefficient)，是指属性中值不匹配的属性的比例

$$dist(x_i, x_j) = \frac{b + c}{a + b + c + d}$$

二元属性的列联表

对象i	对象j			
		1	0	sum
	1	a	b	a+b
	0	c	d	c+d
	sum	a+c	b+d	a+b+c+d

五、距离函数

➤ 非对称布尔属性

- 当布尔属性的一个状态比其他状态更重要时，我们称这个布尔属性是非对称的。通常用1表示那个更为重要的状态，即出现的较少或不太频繁的状态
- 非对称属性中使用最多的距离度量函数是Jaccard距离 (Jaccard coefficient)

$$\text{dist}(x_i, x_j) = \frac{b + c}{a + b + c}$$

- 我们也可以在距离中进行一些变化，增加权重

用二元属性描述的患者记录的关系表

name	gender	fever	cough	test-1	test-2	test-3	test-4
Jack	M	Y	N	P	N	N	N
Jim	M	Y	Y	N	N	N	N
Mary	F	Y	N	P	N	P	N
...	...	...	...	...	...	...	...

$$d(\text{Jack}, \text{Jim}) = \frac{1 + 1}{1 + 1 + 1} = 0.67$$

$$d(\text{Jack}, \text{Mary}) = \frac{0 + 1}{2 + 0 + 1} = 0.33$$

$$d(\text{Jim}, \text{Mary}) = \frac{1 + 2}{1 + 1 + 2} = 0.75$$

五、距离函数

- 符号属性

- 具有多于两个状态(或值)的属性

- 使用最多的距离度量函数也是基于简单匹配距离的

- 给定两个数据点 x_i 和 x_j ，设数据的属性数目为 d ，并且 x_i 和 x_j 之间属性值匹配的属性数目为 q ：

$$\text{dist}(x_i, x_j) = \frac{d - q}{d}$$

五、距离函数

● 文本文档

- 文本文档是由一系列由单词序列组成的句子组成的
 - ▣ 为了简化，在文档聚类中通常把文本文档看作是一组单词的集合(a “bag” of words)，单词的顺序和位置信息被忽略
- 因此，文档可以和普通数据点一样用向量来表示
- 比较两个文档时，通常计算的是两个文档的相似度而不是它们的距离
 - ▣ 使用最多的相似度函数是向量夹角余弦相似度(cosine similarity)

$$\cos(\theta) = \frac{\sum_{i=1}^d (x_i \times y_i)}{\sqrt{\sum_{i=1}^d (x_i)^2} \times \sqrt{\sum_{i=1}^d (y_i)^2}}$$

五、距离函数

- 计算两个句子向量的余弦值来确定两个句子的相似度

句子A: (1, 1, 2, 1, 1, 1, 0, 0, 0)

句子B: (1, 1, 1, 0, 1, 1, 1, 1, 1)

计算过程如下:

$$\begin{aligned}\cos(\theta) &= \frac{1 \times 1 + 1 \times 1 + 2 \times 1 + 1 \times 0 + 1 \times 1 + 1 \times 1 + 0 \times 1 + 0 \times 1 + 0 \times 1}{\sqrt{1^2 + 1^2 + 2^2 + 1^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2} \times \sqrt{1^2 + 1^2 + 1^2 + 0^2 + 1^2 + 1^2 + 1^2 + 1^2 + 1^2}} \\ &= \frac{6}{\sqrt{7} \times \sqrt{8}} \\ &= 0.81\end{aligned}$$

六、数据标准化

- 在使用欧几里德距离时，由于数据标准化能使得各个属性在距离计算过程中具有相同的作用，因此数据标准化有必要
- 考虑下列一对数据点

$x_i(0.1, 20)$ 和 $x_j(0.9, 720)$ 的距离：

$$\text{dist}(x_i, x_j) = \sqrt{(0.9 - 0.1)^2 + (720 - 20)^2} = 700.000457$$

- 这个距离完全由 $(720-20) = 700$ 所主导
- 标准化属性：强制各个属性都在一个相同的范围内变化
 - 区间度量属性、比例属性、符号属性、顺序属性

六、数据标准化

- 区间度量属性(Interval-scaled attributes)

- 这些属性是指数值/连续属性，它们是符合线性标量的实数。比如：年龄、身高、体重、价格等

- 10岁到20岁之间的差异与40岁到50岁之间的差异是相同的

- 主要思想：在整个线性度量空间上相同长度的区间具有相同的重要性

- 两种主要的标准化区间度量属性的方法分别是：范围标准化和z-score标准化。f是一个属性下标。

- 范围标准化：
$$range(x_i) = \frac{x_i - \min_f(x_{if})}{\max_f(x_{if}) - \min_f(x_{if})}$$

六、数据标准化

□ z-score标准化：基于属性的平均值和标准差来对属性进行标准化的

- ✓ z-score标准化数值指出的是属性值从哪个方向远离属性均值以及偏离属性均值的距离，这个值的单位量是属性的标准差。属性 f 的标准差用 s_f 表示。计算公式如下：

$$m_f = \frac{1}{n} (x_{1f} + x_{2f} + \dots + x_{nf})$$

$$s_f = \frac{1}{n} (|x_{1f} - m_f| + |x_{2f} - m_f| + \dots + |x_{nf} - m_f|)$$

z-score:

$$z(x_{if}) = \frac{x_{if} - m_f}{s_f}$$

$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$ // mini-batch mean

$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$ // mini-batch variance

$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$ // normalize

$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i)$ // scale and shift

Batch Normalization

六、数据标准化

● 比例度量属性

- 属性是数值属性，但与区间度量属性不同的是，它的度量不是线性的
- 比如：微生物数量随时间变化的规律可有由 Ae^{Bt} 给出，其中A和B是某个正常数
- 对于这样的属性，标准化的方法有：
 - 采用与区间度量属性一样的处理方法，会在刻度上扭曲，故不推荐
 - 使用对数函数进行转换 $\log(x_{if})$

六、数据标准化

- 符号属性

- 有时需要将符号属性转换成数值属性
- 将符号属性转换成布尔属性
 - 某个符号属性的属性值个数为 v
 - 首先建立 v 个布尔属性分别表示它们
 - 当一个数据实例的该符号属性取了一个特定的属性值时，我们就把该属性值对应的布尔属性的属性值取为1，其余的取0
- 得到的布尔属性可以被当作数值属性（取值0或1）

六、数据标准化

- 符号属性-举例

- 对于符号属性 $fruit$, 有三个取值: Apple, Orange 和 Pear
- 建立三个布尔属性: Apple, Orange 和 Pear
- 如果在数据中某个数据实例的 $fruit$ 属性取值为Apple
 - 则在转换后的数据中, 我们置属性Apple的值为1
 - 属性Orange 和Pear 为 0

六、数据标准化

- 顺序属性

- 与符号属性相似，但它的取值是有特定顺序。比如：

- 年龄属性可能取值： Young, MiddleAge 和 Old，它们是有顺序的
 - 常用的标准化方法： 把它当成区间度量属性，采用与其相同的方法来对这些属性进行标准化

七、混合属性的处理

- 距离函数假定了数据中只有一种数据类型：都是数值属性/符号属性等
- 实际上，数据集中可能含有混合属性
 - 区间度量属性
 - 对称布尔属性
 - 非对称布尔属性
 - 比例度量属性
 - 顺序属性
 - 符号属性

七、混合属性的处理

- 转换成一个类型

- 一种方法就是：

- 先在混合属性中确定一个属性类型作为主导类型
 - 然后把数据集中的其他类型的属性转换成该数据类型

- 比如：一个数据集中大多数属性都是区间度量属性

- 则可将顺序属性、比例属性转换成区间属性
 - 同样，也可以把对称布尔属性转换成区间属性

- 将具有多于两个状态的符号属性或非对称属性转换为区间度量属性是没有意义的

- 在实际情况下，人们还是经常根据某些隐含的顺序信息给这类属性赋予一些数值把它们转换成区间度量函数
 - 如：fruit中的价格来给它们排序

- 前面介绍过：可以把一个符号属性转换成多个对称布尔属性，从而使它被看作是一个区间度量属性

七、混合属性的处理

- 合并距离

➤ 另一种方法就是：

- 首先分别根据各个不同类型的属性计算两个数据点之间的距离
- 然后把这些不同属性的距离结合起来得到一个最终距离

$$\text{dist}(x_i, x_j) = \frac{\sum_{f=1}^d \delta_{ij}^f d_{ij}^f}{\sum_{f=1}^d \delta_{ij}^f}$$

八、聚类算法的选择

- 聚类算法的研究和应用有着悠久的历史，人们设计出了一大批聚类算法
 - 我们仅介绍了其中一小部分算法
- 选择一个在给定应用的数据集上“最好”的算法是一个具有挑战性的工作
 - 每一个聚类算法都有自己的局限性使得它们只能在某种特定的数据分布情况下有较好的聚类结果
 - 要知道应用中的数据集到底是何种分布通常是很难的，甚至是不可能的。事实上，现实应用中的数据集往往并不能完全符合任何一种算法所要求的结构或者分布
 - 另外，还需要如何标准化数据，以此确定一个合适的距离函数和选择其他参数（ k -均值算法中的 k ）

八、聚类算法的选择

- 由于这些复杂度，通常的做法是：
 - 执行不同的算法，使用不同的距离函数和不同的参数设置
 - 并对结果进行仔细的分析和比较
- 结果的理解需要同时建立在对原始数据的深刻理解和对于所使用的算法的认识之上

九、聚类的评估

- 对于聚类算法的评估是很困难的
 - 我们不知道在某个数据集上的正确聚类是什么
- 有些聚类评估方法
 - 用户验证
 - 理解聚类结果的聚类中心
 - 依据规则
 - 文本文档的聚类，用户很容易阅读这些文档

九、聚类的评估

● 聚类评估——真实数据

- 使用一些有标记的数据（用分类数据集来评估聚类算法）
- 假设：每一个类别对应一个聚类
- 聚类操作之后，可以根据结果构造一个混淆矩阵。从这个矩阵中，可以计算熵、纯度、查准率、查全率和F-score值来进行评估聚类的质量
 - 设数据集 D 中的类别集合 $C = (c_1, c_2, \dots, c_k)$ ，聚类算法也生成 k 个聚类，这 k 个聚类把数据集 D 划分成 k 个两两不相交的集合 $D_1, D_2, \dots, D_k$

九、聚类的评估

● 熵

对于每个聚类，我们可以按下列方法度量它们的熵。

$$entropy(D_i) = - \sum_{j=1}^k Pr_i(c_j) \log_2 Pr_i(c_j)$$

其中， $Pr_i(c_j)$ 是聚类 i 或 D_i 中属于类别 c_j 的数据点所占的比例。整个聚类结果（即所有聚类）的熵总和为：

$$entropy_{total}(D) = \sum_{i=1}^k \frac{|D_i|}{|D|} \times entropy(D_i)$$

九、聚类的评估

● 纯度

这个度量方法度量的的是一个聚类中仅包含一个类别的数据的程度。每个聚类的纯度可以按如下方法计算：

$$purity(D_i) = \max_j (\Pr_i(c_j))$$

整个聚类结果的纯度总和为：

$$purity_{total}(D) = \sum_{i=1}^k \frac{|D_i|}{|D|} \times purity(D_i)$$

九、聚类的评估

● 举例

- 假设有一个包含来自科学、体育和政治三个主题的900篇文档的文本集合D，并且每个文档都被标记为其中一个主题。我们在这个数据集上用聚类方法来发现三个聚类
- 主题标识在聚类过程中是不被使用到的，聚类完成以后，来评估这个聚类算法的有效性

Cluster	Science	Sports	Politics		Entropy	Purity
1	250	20	10		0.589	0.893
2	20	180	80		1.198	0.643
3	30	100	210		1.257	0.617
Total	300	300	300		1.031	0.711

九、聚类的评估

- 真实数据评估法的讨论

- 经常用来比较不同聚类算法的效率
- 用于聚类的真实数据是没有类别标识
 - 因此，即使算法在某些已标注的数据集上有很好的聚类结果，但并不能保证这个算法在没有类别标识的实际应用数据上也会表现良好
- 事实上，在一些已标注数据集上的良好表现能够使我们对于算法的质量有一定信心
- 这种评估方法被称作是基于外部数据或信息的

九、聚类的评估

- 基于内部信息的聚类评估

- 聚类内紧密度

- 数据点和聚类中心的相似度
- 通常使用Sum of squared error (SSE)来进行测量

- 聚类间分离度

- 不同的聚类中心之间的差异度
- 在很多应用中，专家的意见仍然是最为重要的

$$S_w^{(i)} = \frac{1}{N} \sum_{k=1}^{N_i} (X_k^{(i)} - m^{(i)})(X_k^{(i)} - m^{(i)})^T$$

$$S_B^{(ij)} = (m^{(i)} - m^{(j)})(m^{(i)} - m^{(j)})^T$$

九、聚类的评估

● 间接评估

- 在一些应用中，聚类操作并不是主要的任务，而是作为另一个任务的辅助
- 我们可以利用主要任务的功率来衡量到底哪个聚类算法对于这个任务更合适
- 比如：在Web使用信息挖掘的应用中，主要的任务是给网上客户提供图书的推荐功能
 - 如果能够根据客户的兴趣和以前历史购买记录来对顾客进行聚类，我们可能会提供一个更好的推荐服务
 - 我们就可以通过它们对于推荐服务提供的帮助上来评估这些聚类技术
 - 这里，我们假设推荐服务的结果可以很可靠地被评价

本章小结

- 聚类研究有着很长的历史，并且有着大量的工作
 - 有很多聚类算法
 - 每一年都有新的进展
- 本章仅介绍了一些广泛使用的算法，还有其他很多很多算法
- 聚类评估很难，但在实际应用中却很有用。这也是为什么每一年还会研究出一系列聚类算法的原因

多类划分的评价

- 通过混淆矩阵来求多分类问题的精确率、召回率和F1值
 - 若共有F类，则定义一个 $F \times F$ 的混淆矩阵M，其内部元素 M_{ij} 表示类i的节点被分为j的个数

$$Precision_i = \frac{M_{i,i}}{\sum_j M_{j,i}}$$

$$Recall_i = \frac{M_{i,i}}{\sum_j M_{i,j}}$$

$$F1_i = \frac{2Precision_i \times Recall_i}{Precision_i + Recall_i}$$

$$\left\{ \begin{array}{l} Precision = \frac{\sum_{i=1}^F Precision_i}{F} \\ Recall = \frac{\sum_{i=1}^F Recall_i}{F} \\ F1 = \frac{\sum_{i=1}^F F1_i}{F} \\ Accuracy = \frac{\sum_i M_{i,i}}{\sum_i \sum_j M_{i,j}} \end{array} \right.$$

聚类——作业

假设挖掘任务是将如下的八个点（用 (x,y) 代表位置）：

$A_1(2,10)$, $A_2(2,5)$, $A_3(8,4)$, $B_1(5,8)$, $B_2(7,5)$, $B_3(6,4)$, $C_1(1,2)$, $C_2(4,9)$

1、要求聚类为三个簇，假设初始我们选择分别 $A_1B_1C_1$ 为每个簇的中心，用K-均值算法给出：

1) 第一轮执行后的三个簇中心

2) 最后的三个簇

2、要求聚类为两个簇，采用层次聚类中的合并聚类方法完成聚类。